Fitra System Requirement Specifications (SRS)

Version <1.1>

Project Manager (Overall delivery of product)	Gary Quah YiKai
Lead Developer (Overall technical lead, responsible for technical aspects of product release)	Matthew Wong
QA Manager (Overall product and process quality, implementation of QA processes)	Lau Jing Jie
QA Engineer (Devise test plans, conduct tests)	Lee Zheng Wee
Software Configuration Manager (SCM)	Rusyaidi
Release Engineer/Manager (Create baselines and build and integrate changes for delivery)	Quah Yew Yi

Revision History

Revision Number	Date	Primary Author(s)	Comments
0.1	Sept 10th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	First edit
0.2	Sept 16th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Second Edit
1.0	Sept 23rd, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Final Modifications
1.1	Nov 4th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Revised the two-service architecture; unified Home/G-1 daily questionnaire, Singapore-date streak and once-daily XP rules; clarified server-side image conversion and authoritative meal totals; specified the 10% training-volume threshold.

Table of contents

Revision History **2**

1. Introduction **5**

1.1. Problem statement 5

1.2. Background 5

1.3. Purpose 5

1.4. Intended Audience 6

1.5. Objectives 6

1.6. Scope 6

1.7. Overall Description 6

2. Investigation & analysis methodology **7**

2.1. System Investigation 7

2.2. Analysis Methodology 8

2.2.1. System analysis and requirements specification 8
2.2.1.1. Perform an analysis of the problem using object-oriented techniques 8
2.2.1.3. Login Page 9
2.2.1.4. Friend page 13
2.2.1.5. Home page 16
2.2.1.6. Settings page 21
2.2.1.7. Chatbot Page 23
2.2.1.8. Training Routine Page 24
2.2.1.9. Meal analysis page 31
2.2.1.10. Gamification page 43
3. Constraints 48
3.1. Scalability 48
3.2. Data and Function Mapping 48
3.3. Proprietary hardware and software 48
3.4. Batch updates vs. (close) Real-time updates 48
3.5. Project Schedule 49
4. Operational Requirements 52
4.1. Help Desk Support 52
4.2. Application Services and Technical support 52
4.3. Administration Features 52
4.4. System Interface independent of VRU 52
4.5. System routine back up 53
4.6. Audit Trail 53
5. Functional Requirements 54
5.1. Login/Registration page 54
5.2. Friend page 54
5.3. Home page 54
5.4. Settings page 54
5.5. Meal analysis page 55
5.6. Chatbot page 56
5.7. Training routine page 57
5.8. Gamification page 59
6. Non-functional requirements 59
6.1. Performance 59
6.2. Scalability 59
6.3. Maintainability 59
6.4. User experience 60
6.5. Security 60
7. Input Requirements 61

7.1. User identifier key and user access 61
7.2. Health and training data 61
7.3. Action Codes 61
8. Process Requirements 62
8.1. User Authentication 62
8.2. Data integrity 62
8.3. Data Validation and Error Handling 62
8.4. API and Database Communication 62
8.5. Performance and Availability 62
8.6. Data Security 62
9. Output Requirements 63
9.1. Transaction summary and confirmation 63
9.2. Exception reports 63
9.3. Registration Reports and summaries 63
10. Hardware Requirements 64
10.1. Network 64
10.2. Client Computers 64
10.3. Server Infrastructure 64
10.4. Production support Systems 64
11. Software Requirements 65
11.1. Operating System 65
11.2. Client Application 65
11.3. Frontend 65
11.4. Backend 65
11.5. Database and Authentication Services 65
12. Deployment Requirements 66
12.1. System architecture overview 66
12.2. Hardware requirements 67
12.3. Deployment prerequisites & configuration 68
12.4. Deployment artifacts & procedure 69

1. Introduction

1.1. Problem statement

Many people want to use technology to improve their health, but fragmented and generic tracking tools often destroy their daily consistency. Users desperately need a centralized platform. Right now, applications fail to integrate human biological metrics seamlessly with external athletic telemetry from services like Strava or Samsung Health. Tedious manual nutrition logging frustrates users. Static training routines completely ignore daily readiness, which creates a massive demand for AI-driven meal image analysis and dynamic workout generation. Users also require intelligent contextual coaching. An AI conversational interface can evaluate a user's specific energy expenditure and history to provide personalized guidance and reduce psychological friction. Low user retention ruins fitness programs. To build long-term motivation, the application must implement structured gamification features like experience points, dynamic peer leaderboards, shared user challenges, and interactive daily check-ins.

1.2. Background

People of all ages want better health. They actively seek out modern technology to help them easily manage their complex fitness levels and track their daily nutritional habits. They need personalized planning assistance. Our team is designing a smart fitness tracker to automatically analyze personal health data and daily food consumption using artificial intelligence. We create custom training schedules. By demonstrating strong software engineering skills, we intend to build an effective web application that generates personalized meal plans for any user.

1.3. Purpose

The primary purpose of the project is to develop a smart fitness tracker web application that provides personalised training scheduling and meal planning.

The application aims to assist users in planning their workouts and nutrition based on their personal health, fitness and daily food consumption data by leveraging Artificial Intelligence (AI) and data analytics to provide personalised recommendations.

1.4. Intended Audience

The target audience for this application includes people of different ages and fitness levels who are keen on improving their health and lifestyles through technology-assisted fitness and nutrition management.

1.5. Objectives

The main objectives of this project are:

1. To design and develop a fitness tracker incorporating Artificial Intelligence (AI) and data analytics
2. To demonstrate sound software engineering skills

1.6. Scope

The scope of this project includes:

1. Development of a full-stack web application consisting of both frontend and backend components.
2. Frontend implementation for user interaction, input handling, and result visualisation.
3. Backend implementation for business logic, data management, and integration with Gen-AI services or other external API services.

1.7. Overall Description

The system uses two application services: a Next.js frontend running on Node.js and one Python FastAPI backend running on Uvicorn. Next.js handles pages, navigation, sessions and UI interactions. FastAPI contains the authenticated feature APIs, application rules, data access, readiness scoring and external API integrations.

Readiness inference runs within FastAPI; meal analysis and AI coaching call the external Google Gemini API. Supabase provides managed authentication and PostgreSQL storage.

A user registers an account and enters their metrics, activity level and fitness goal. The system calculates their Metabolic Rate, Total Daily Energy Expenditure and daily macronutrient targets. Health and activity data enters the system in three ways: by importing an export file from an external health service, by linking a Strava account, or by entering a session manually. The home page presents this data as training and nutrition trends, and a daily check-in records 1–5 ratings for muscle fatigue, mood and yesterday's diet adherence; sleep hours are entered separately on the Training page. A readiness model scores each day from this data and classifies it as deload, light or push, which drives the training plan and the recommended workout. The Google Gemini API estimates the calories and macronutrients of a meal from a photograph, and powers a chatbot that answers questions using the user's own records.

The application runs in any browser and requires no proprietary hardware; a wearable is optional and only supplies additional heart rate data. Activity data retrieved from the Strava API is used for display only, in accordance with the Strava API Policy.

2. Investigation & analysis methodology

2.1. System Investigation

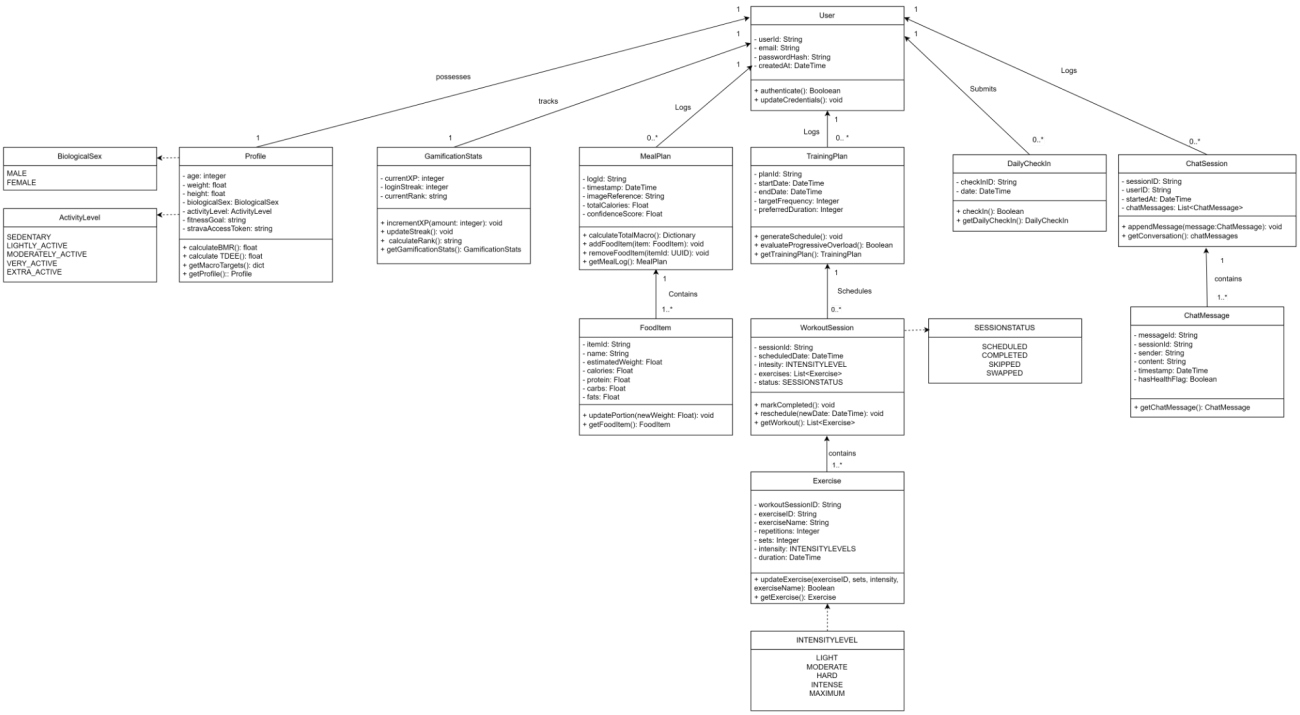
The team investigates how current users actually manage both their training and nutrition. Recorded metrics like height, weight, resting heart rate are usually held in a wearable device application, while recorded activities are held in other services such as Strava, and lastly nutrition is logged in another application. Although these tools present raw data to the user, it doesn't provide an actionable plan for them, so the user is left to interpret the data.

2.2. Analysis Methodology

2.2.1. System analysis and requirements specification

2.2.1.1. Perform an analysis of the problem using object-oriented techniques

An external view of the enterprise model of the fitness application includes user data records, training information,meal logs, gamification stats e.t.c will be developed using Unified Modeling Language (UML). This System Requirement Specifications documents will form part of the documentation for the project.

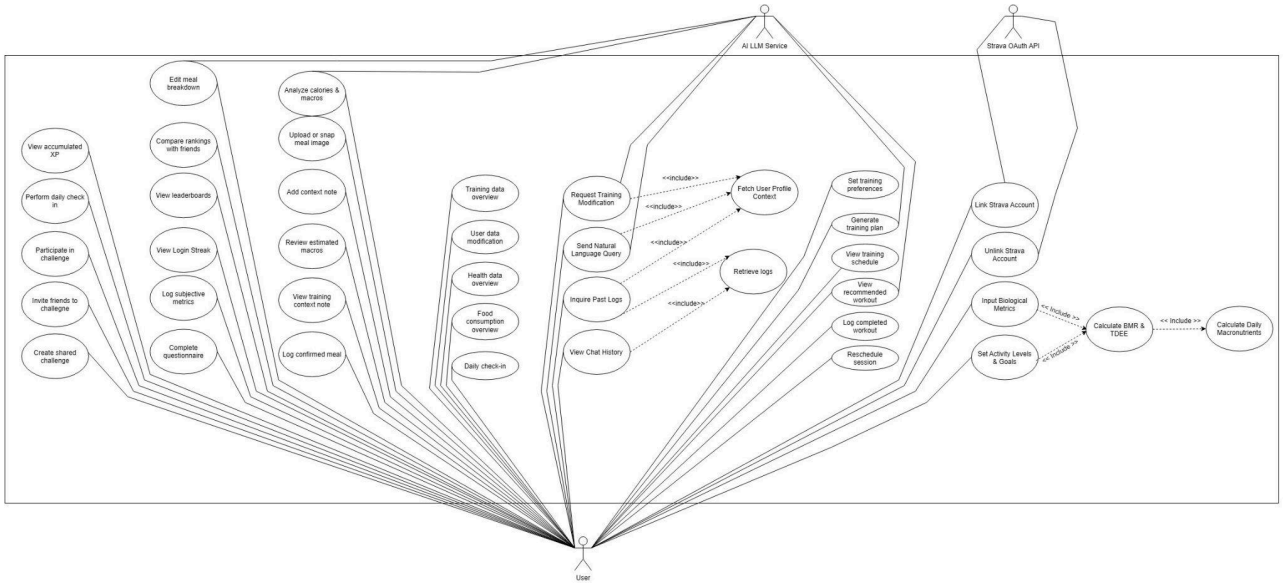


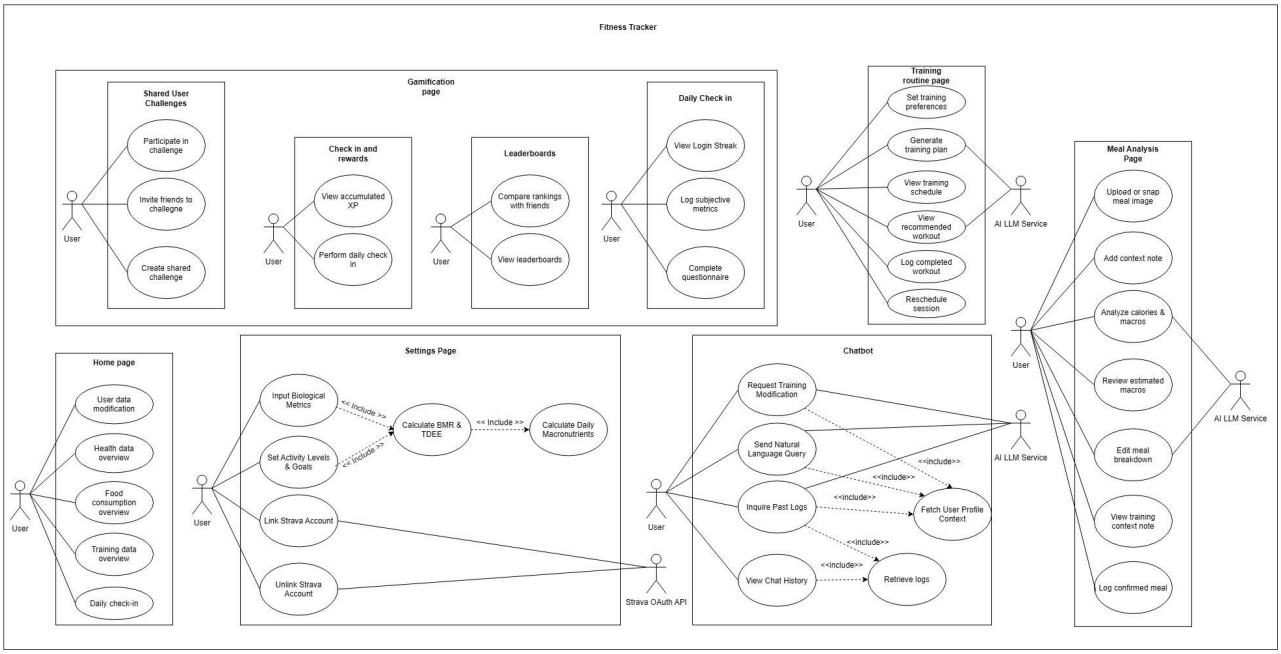
2.2.1.2. Scope and Limitations

2.2.1.2.1. Third-Party API Limits

This application relies heavily on many external APIs, specifically Gemini and Strava for many of our core system components. The application is constrained by the rate limit and token expiration policies managed by the

third-party services. Our application must handle timeouts and token refresh on the backend to ensure a good user experience.





2.2.1.3. Login Page

2.2.1.3.1. Use case: User Registration

Use Case ID	L - 1
Use Case Name	User Registration
Created By / Updated By	Rusyaidi
Date Created / Updated	12/09/26
Actors	User
Description	Allows a new user to create an account for the Health Tracker system.
Trigger	User selects the registration option on the login page.
Preconditions	User does not have an existing account.
Postconditions	A new user account is created successfully and the user can log in to the system.
Normal Flow	1. User selects the registration option. 2. System displays the registration form. 3. User enters the required information. 4. User submits the registration form. 5. System validates the information. 6. System creates the user account. 7. System informs the user that registration was successful.
Alternative Flows	A1. User returns to the login page without completing registration
Exceptions	E1. Required information is missing or invalid. E2. The email address is already registered. E3. Account creation fails.
Priority / Frequency	High / Once per user
Special Requirements	User account information shall be securely stored.

2.2.1.3.2. Use case: User Login

Use Case ID	L - 2
Use Case Name	User Login
Created By / Updated By	Rusyaidi
Date Created / Updated	12/09/26
Actors	User
Description	Allows registered users to authenticate and access the Health Tracker system.
Trigger	User enters their login credentials and selects the login option.
Preconditions	User has a registered account.
Postconditions	User is successfully authenticated and granted access to the system.
Normal Flow	1. User enters their email address and password. 2. User selects the login button. 3. System validates the provided credentials. 4. System authenticates the user. 5. System grants the user access to the system.
Alternative Flows	A1. User selects the registration option and proceeds to User Registration.
Exceptions	E1. Email address or password is incorrect. E2. Required login information is missing. E3. Authentication fails.
Priority / Frequency	High / Every time a user accesses the system
Special Requirements	User credentials shall be securely handled and authenticated.

2.2.1.3.3. Use case: User Logout

Use Case ID	L - 3
Use Case Name	User Logout
Created By / Updated By	Rusyaidi
Date Created / Updated	12/09/26
Actors	User
Description	Allows an authenticated user to log out of the Health Tracker system.
Trigger	User selects the logout option.
Preconditions	User is currently logged in.
Postconditions	User is logged out and can no longer access authenticated pages.
Normal Flow	1. User selects the logout option. 2. System terminates the user's session. 3. System redirects the user to the login page.
Alternative Flows	None.
Exceptions	E1. Logout operation fails.
Priority / Frequency	High / Every time a user chooses to log out.
Special Requirements	The user's authenticated session shall be securely terminated.

2.2.1.4. Friend page

2.2.1.4.1. Use case: View Friends

Use Case ID	F - 1
Use Case Name	View Friends
Created By / Updated By	Rusyaidi
Date Created / Updated	12/09/26
Actors	User
Description	Allows an authenticated user to view their current friends.
Trigger	User navigates to the Friends page.
Preconditions	User is logged in.
Postconditions	The user's current friends are displayed.
Normal Flow	1. User navigates to the Friends page. 2. System retrieves the user's friend information. 3. System displays the user's current friends.
Alternative Flows	A1. User has no existing friends. The system displays an appropriate message.
Exceptions	E1. Friend information cannot be retrieved. E2. System encounters an error while loading the Friends page.
Priority / Frequency	High / Whenever the user accesses the Friends page
Special Requirements	Friend information shall be displayed accurately and only to authorised users.

2.2.1.4.2. Use case: Add Friend

Use Case ID	F - 2
Use Case Name	Add Friend
Created By / Updated By	Rusyaidi
Date Created / Updated	12/09/26
Actors	User
Description	Allows an authenticated user to search for and add another user as a friend.
Trigger	User chooses to add a friend from the Friends page.
Preconditions	User is logged in.
Postconditions	The selected user is added to the user's friend list.
Normal Flow	1. User enters a search query for another user. 2. System displays matching users. 3. User selects the desired user. 4. User selects the option to add the user as a friend. 5. System adds the selected user to the user's friend list. 6. System provides confirmation to the user.
Alternative Flows	A1. User does not find the desired user and performs another search. A2. User cancels the operation before adding the friend.
Exceptions	E1. No users match the search query. E2. The selected user cannot be added. E3. System encounters an error while updating the friend list.
Priority / Frequency	High / As required by the user
Special Requirements	The system shall prevent invalid or duplicate friend relationships.

2.2.1.4.3. Use case: Remove Friend

Use Case ID	F - 3
Use Case Name	Remove Friend
Created By / Updated By	Rusyaidi
Date Created / Updated	12/09/26
Actors	User
Description	Allows an authenticated user to remove an existing friend from their friend list.
Trigger	User selects the option to remove a friend.
Preconditions	User is logged in and has at least one existing friend.
Postconditions	The selected friend is removed from the user's friend list.
Normal Flow	1. User views their friend list. 2. User selects the friend they want to remove. 3. User selects the remove friend option. 4. System removes the selected friend. 5. System updates the displayed friend list.
Alternative Flows	A1. User cancels the removal operation.
Exceptions	E1. The selected friend cannot be removed. E2. System encounters an error while updating the friend list.
Priority / Frequency	Medium / As required by the user
Special Requirements	Only the authenticated user shall be able to modify their own friend list.

2.2.1.5. Home page

2.2.1.5.1. Use case: User data modification

Use Case ID	H -1
Use Case Name	User data modification
Created By / Updated By	Gary Quah / Gary Quah
Date Created / Updated	10/09/2026
Actors	User
Description	The user must be able to update their health information.
Trigger	The user clicks on the “Edit health data” button.
Preconditions	1. The internet connection is stable. 2. The website is launched.
Postconditions	1. A pop-up interface will appear with the user's current health details as input fields for the user to edit.
Normal Flow	1. The user clicks on the “Edit health” button 2. A pop-up interface appears with the user's current health details as input fields for the user to edit. 3. The Application updates user information in the database along with the user interface corresponding to the data modified, such as height and weight.
Alternative Flows	Nil
Exceptions	System failed to update health data: 1. Show error “Unable to update your data, contact an administrator if you think this is unintentional”
Priority / Frequency	Medium, occasionally if the user finds their imported data to be wrong or does not use the health data importing service.

Special Requirements	Nil

2.2.1.5.2. Use case: Health data overview

Use Case ID	H -2
Use Case Name	Health data overview
Created By / Updated By	Gary Quah / Gary Quah
Date Created / Updated	10/09/2026
Actors	User, external health services
Description	The user must be able to view their health data in the form of metrics..
Trigger	The user is at the home page
Preconditions	1. The internet connection is stable. 2. The website is launched.
Postconditions	1. User interface will display the users health data
Normal Flow	1. The user is at the home page 2. The user views their health data
Alternative Flows	Nil
Exceptions	Nil
Priority / Frequency	High, every time the user accesses the application
Special Requirements	Nil

2.2.1.5.3. Use case: Training data overview

Use Case ID	H -3

Use Case Name	Training data overview
Created By / Updated By	Gary Quah / Gary Quah
Date Created / Updated	10/09/2026
Actors	User
Description	The user must be able to view their training data for a period of time, such as the most recent training data or daily training data.
Trigger	The user is at the home page, and is looking at the training overview segment of the dashboard.
Preconditions	1. The internet connection is stable. 2. The website is launched. 3. The user is looking at the training overview segment.
Postconditions	1. User interface will display the users training data such as total calories burnt, volume, average heart rate, Rate of Perceived Exertion and total duration.
Normal Flow	1. The user is at the home page 2. The user views their training data
Alternative Flows	Nil
Exceptions	Nil
Priority / Frequency	High, every time the user accesses the application
Special Requirements	Nil

2.2.1.5.4. Use case: Food consumption overview

Use Case ID	H -4
Use Case Name	Food consumption overview

Created By / Updated By	Gary Quah / Gary Quah
Date Created / Updated	10/09/2026
Actors	User
Description	The user must be able to view their consumed calories for a period of time, such as daily or weekly.
Trigger	The user is at the home page, looking at the food consumption segment of the dashboard.
Preconditions	1. The internet connection is stable. 2. The website is launched. 3. The user is looking at the food consumption segment of the dashboard.
Postconditions	1. User interface will display the users consumed calories data such as total calories consumed and target calories.
Normal Flow	1. The user is at the home page 2. The user views their food consumption data
Alternative Flows	Nil
Exceptions	Nil
Priority / Frequency	High, every time the user accesses the application
Special Requirements	Nil

2.2.1.5.5. Use case: Daily check-in

Use Case ID	H -5
Use Case Name	Daily check-in
Created By / Updated By	Gary Quah / Gary Quah

Date Created / Updated	10/09/2026
Actors	User
Description	The user must be able to view their check-in status, check-in streak and be directed to check-in if the user has yet to do so.
Trigger	The user clicks the “Check-in” button.
Preconditions	1. The internet connection is stable. 2. The website is launched.
Postconditions	1. The Home button opens the shared G-1 questionnaire without changing the streak or XP. After a successful questionnaire submission, the Home calendar and Gamification page reflect the same saved check-in, streak and XP.
Normal Flow	1. The user is on the home page 2. The user clicks the “Check-in” button 3. The user is directed to the daily questionnaire on the leaderboard page (G-1), completes all three ratings and submits. The system records the check-in and applies the shared streak and XP rules only after successful submission.
Alternative Flows	If the user has already checked in today, Home displays “Checked in” and disables the button. G-1 displays the completed state; a repeat submission is rejected without another streak increment or XP award.
Exceptions	Nil
Priority / Frequency	High, every time the user accesses the application
Special Requirements	Home and G-1 use the same saved daily check-ins. A day is 00:00–23:59 Singapore time (UTC+8); only a completed questionnaire counts toward the check-in streak and daily XP.

2.2.1.6. Settings page

2.2.1.6.1. Use Case: Settings + TDEE

Use Case ID	S - 1
Use Case Name	Settings + TDEE
Created By / Updated By	Jing Jie / Jing Jie
Date Created / Updated	11/09/2026
Actors	User
Description	The user inputs their biological metrics and activity levels to calculate their BMR, TDEE and target macronutrients.
Trigger	The user clicks the “Save Profile” button on the settings page
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated
Postconditions	1. Database is updated with new biological metrics and fitness goals. 2. The application calculates and display the updated BMR, TDEE and macronutrients target
Normal Flow	1. User navigates to the Settings page 2. The user inputs their age, weight, height, and sex 3. The user selects their activity level and fitness goals (Deficit, Maintenance, surplus) 4. The user clicks on “Save Profile” 5. Application backend calculates the BMR, TDEE using Mifflin-St Jeor equation 6. Application backend calculates the daily macronutrient distribution 7. Application saves the data and refreshes the UI with the newly created metrics
Alternative Flows	Nil

Exceptions	1. Invalid data input (negative weight or missing fields) a. Show error “Please enter valid biological metrics” 2. Database connection failure a. Show error “Failed to save profile, please try again later”
Priority / Frequency	High, infrequent updated only when user first create account or goal change
Special Requirements	Nil

2.2.1.6.2. Link Strava

Use Case ID	S - 2
Use Case Name	Link Strava
Created By / Updated By	Jing Jie / Jing Jie
Date Created / Updated	11/09/2026
Actors	User, Strava OAuth API
Description	The user links their strava account to authorise the application to fetch past exercise activity history
Trigger	The user clicks the “Link Strava” button on the settings page
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated
Postconditions	1. A persistent Strava access token is stored securely in the database 2. A Refresh token is stored securely in the database 3. Settings page UI is updated indicating that the user has linked their account

Normal Flow	1. User navigates to the Settings page 2. The user clicks the “Link Strava” button 3. The application redirects the user to the official Strava OAuth authorization portal 4. The user logs into their Strava account and clicks “Authorize” 5. Strava redirects back to the application callback URL together with the authorization code 6. The backend exchanges the code for tokens and saves them to the database 7. The application updates the Setting page UI
Alternative Flows	1. User clicks on “Cancel” on the Strava authorization portal 2. Redirects user back to the Settings page without linking account
Exceptions	1. Authorization / token exchange fails or StravaAPI times out 2. Show error “Failed to authenticate with Strava. Please try again later”
Priority / Frequency	High, Once per user
Special Requirements	Nil

2.2.1.7. Chatbot Page

2.2.1.7.1. Chatbot AI Query

Use Case ID	C - 1
Use Case Name	Chatbot AI Query
Created By / Updated By	Jing Jie / Jing Jie
Date Created / Updated	11/09/2026
Actors	User, Gemini API
Description	The user interacts with AI to ask for fitness or nutritional questions and receive a context-aware response based on their historical and physiological data

Trigger	The user inputs their query and clicks the “Send” button in chatbot page.
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated
Postconditions	1. Chat session history is updated on the UI 2. Displays both user’s prompt and AI response
Normal Flow	1. The user types a natural language query into the chat input field and clicks “Send” 2. Application displays the user’s message in the chat history and render a loading indicator waiting for AI response 3. Backend retrieves the relevant user’s physiological data 4. Backend injects the data along with users query and sends the payload to Gemini API 5. Gemini API processes the prompt and returns a response 6. Application remove the loading indicator and appends the response into the chat history
Alternative Flows	Nil
Exceptions	1. Gemini API timeout 2. Show error “Failed to save profile, please try again later”
Priority / Frequency	Medium, High frequency during active user session
Special Requirements	Nil

2.2.1.8. Training Routine Page

2.2.1.8.1. Setting Training Preferences

Use Case ID	T - 1
Use Case Name	Setting Training Preferences
Created By / Updated By	Zheng Wee / Zheng Wee

Date Created / Updated	13/09/2026
Actors	User
Description	The user sets their training preferences so that the application can generate a training plan that fits their availability.
Trigger	The user clicks the “Save Preferences” button on the training routine page
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated 4. The user has saved their profile on the settings page
Postconditions	1. Database is updated with the user’s training preferences 2. The “Generate Plan” option becomes available on the training routine page
Normal Flow	1. User navigates to the Training Routine page 2. The user selects the number of training days per week 3. The user selects their preferred session duration 4. The user selects their preferred activity types (run, gym, cycling, bodyweight) 5. The user clicks on “Save Preferences” 6. Application backend validates the selected values 7. Application saves the training preferences to the database and enables the “Generate Plan” option
Alternative Flows	1. The user already has saved training preferences 2. The form is pre-filled with the saved values and the existing record is updated instead
Exceptions	1. No activity type selected or values out of range 2. Show error “Please select at least one activity type and a valid weekly training frequency” 3. Database connection failure 4. Show error “Failed to save preferences, please try again later”
Priority / Frequency	High, infrequent updates only when the user first sets up their plan or their availability changes

Special Requirements	Nil

2.2.1.8.2. Generate Training Plan

Use Case ID	T - 2
Use Case Name	Generate Training Plan
Created By / Updated By	Zheng Wee / Zheng Wee
Date Created / Updated	13/09/2026
Actors	User
Description	The user generates a multi-week training plan based on their profile, fitness goal, training preferences and past training history.
Trigger	The user clicks the “Generate Plan” button on the training routine page
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated 4. The user has saved their training preferences 5. The user has saved their biological metrics and fitness goal on the settings page
Postconditions	1. Database is updated with a new training plan and its planned sessions 2. The training routine page displays the generated plan in the schedule view
Normal Flow	1. User navigates to the Training Routine page 2. The user selects the plan duration, which defaults to 4 weeks 3. The user clicks on “Generate Plan” 4. Application backend retrieves the user’s profile, fitness goal, training preferences and past training history 5. Application backend calculates a baseline weekly training volume from the user’s training history

	6. Application backend distributes the sessions across each week and applies progressive overload within the safe weekly increase limit 7. Application saves the plan to the database and displays it in the schedule view
Alternative Flows	1. The user already has an active training plan 2. Application prompts the user to confirm before replacing the remaining sessions of the current plan 3. The user has no past training history 4. Application generates a conservative starter plan and marks it as provisional
Exceptions	1. The selected training days cannot fit the requested number of sessions 2. Show error “Your training preferences cannot fit the selected plan, please adjust your weekly availability” 3. Database connection failure 4. Show error “Failed to generate your plan, please try again later”
Priority / Frequency	High, once every plan cycle of about 4 weeks
Special Requirements	The generated plan shall not increase weekly training volume beyond the configured safe limit.

2.2.1.8.3. Daily Recommended Workout

Use Case ID	T - 3
Use Case Name	Daily Recommended Workout
Created By / Updated By	Zheng Wee / Zheng Wee
Date Created / Updated	13/09/2026
Actors	User
Description	The user views the workout recommended for the current day, which is adjusted to their readiness score, and chooses to accept, swap or skip it.

Trigger	The user is at the training routine page
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated 4. The user has an active training plan 5. A readiness score has been calculated for the current date
Postconditions	1. The recommended workout for the day is displayed together with the readiness score behind it 2. The user's response of accept, swap or skip is saved against the planned session
Normal Flow	1. The user is at the Training Routine page 2. Application backend retrieves the planned session for the current date and the readiness score for the day 3. Application backend lowers the planned intensity where the readiness classification is lower than the planned intensity 4. Application displays the recommended workout with its activity type, target volume, target intensity and estimated duration 5. Application displays the readiness score and the reason for any adjustment 6. The user clicks on "Accept" to start the session
Alternative Flows	1. The user clicks on "Swap" 2. Application offers alternative workouts of the same intensity for the user to choose from 3. The user clicks on "Skip" 4. Application marks the session as missed 5. The current date is a rest day in the plan 6. Application displays the rest day and suggests optional active recovery
Exceptions	1. No readiness score is available for the current date 2. Application displays the planned session without an intensity adjustment and shows a message explaining why 3. The user has no active training plan 4. Application displays a prompt directing the user to generate a plan

Priority / Frequency	High, every time the user accesses the training routine page
Special Requirements	A day classified as deload shall never display a high intensity session.

2.2.1.8.4. Log Completed Workout

Use Case ID	T - 4
Use Case Name	Log Completed Workout
Created By / Updated By	Zheng Wee / Zheng Wee
Date Created / Updated	13/09/2026
Actors	User
Description	The user records the outcome of a training session so that it is added to their training history and used in future readiness scoring.
Trigger	The user clicks the “Mark as Complete” button on the current session
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated 4. The user has a planned or accepted session for the current date
Postconditions	1. Database is updated with the completed session 2. The training history and the schedule view are updated 3. The readiness score for the next day is recalculated using the new session
Normal Flow	1. The user clicks on “Mark as Complete” on the current session 2. Application displays a form pre-filled with the planned activity type, volume and duration 3. The user edits the values to match what they actually completed 4. The user optionally enters their perceived exertion and average heart rate 5. The user clicks on “Save Session” 6. Application backend validates the entered values

	7. Application saves the completed session to the database and updates the schedule view
Alternative Flows	1. The user logs a session that is not part of their plan 2. Application saves it as an additional session that still counts towards their training load 3. The user completed only part of the session 4. Application saves the session as partially completed
Exceptions	1. Invalid data input (negative or missing volume or duration) 2. Show error “Please enter a valid session duration and volume” 3. Database connection failure 4. Show error “Failed to save your session, please try again later”
Priority / Frequency	High, once on every training day
Special Requirements	Only sessions recorded in the application shall count towards the user’s training load.

2.2.1.8.5. View Training Schedule

Use Case ID	T - 5
Use Case Name	View Training Schedule
Created By / Updated By	Zheng Wee / Zheng Wee
Date Created / Updated	13/09/2026
Actors	User
Description	The user must be able to view their planned and completed training sessions in a weekly or monthly calendar.
Trigger	The user is at the training routine page
Preconditions	1. The internet connection is stable 2. The website is launched 3. The user is authenticated 4. The user has an active training plan

Postconditions	1. The user interface displays the planned and completed sessions for the selected period
Normal Flow	1. The user is at the Training Routine page 2. Application backend retrieves the planned sessions and completed sessions for the selected period 3. Application displays each day with its activity type, target volume and target intensity 4. Application distinguishes completed, partially completed, missed and upcoming sessions 5. The user switches between the weekly and monthly view, or moves to another period
Alternative Flows	1. The user selects an individual session 2. Application displays the full details of that session, including the values recorded on completion 3. The selected period is beyond the end of the active plan 4. Application displays an empty period and offers to generate the next plan
Exceptions	1. The user has no active training plan 2. Application displays a message directing the user to generate a plan 3. Session data cannot be retrieved 4. Show error “Unable to load your training schedule, please try again later”
Priority / Frequency	High, every time the user accesses the training routine page
Special Requirements	Session states shall be distinguishable without relying on colour alone.

2.2.1.9. Meal analysis page

2.2.1.9.1. Upload Meal Image

Use Case ID	MA - 1
Use Case Name	Upload Meal Image
Created By / Updated By	Matthew Wong

Date Created / Updated	13/09/2026
Actors	User
Description	The user uploads a photo of their meal from their local storage
Trigger	The user clicks the “Upload Image”/ file section dropzone on the Meal analysis page
Preconditions	1. The user is logged into their account 2. The user is on the Meal Analysis page
Postconditions	The meal photo is loaded into memory and staged for nutritional analysis. A preview is displayed where supported by the browser; otherwise, the selected filename is displayed
Normal Flow	3. The user taps or clicks on the upload/camera dropzone area. 4. The system triggers the device's native file explorer or camera capture dialog. 5. The user selects an image file from local storage. 6. The system validates the file format (JPEG, PNG, WEBP, HEIC/HEIF) and verifies that file size is at most 10 MB. 7. The system renders the image preview where the browser supports it; otherwise, it displays the selected filename and keeps the image available for analysis, including HEIC/HEIF. 8. The system enables the "Analyze Meal" action button.
Alternative Flows	**4a. Exceeded File Size:** 1. If file size exceeds 10 MB, the system prompts: *"Image size exceeds limit. Please select a file under 10MB."* 2. The file is rejected, and the dropzone resets to its initial state. **4b. Image Replacement:** 1. The user clicks on the image preview container to select a different photo. 2. The previous preview is replaced with the newly selected image.

Exceptions	**E1. Unsupported File Format:** 1. The selected file is not a supported image format (e.g., PDF, GIF, BMP). 2. The system alerts the user: *"Unsupported file format. Please upload a valid JPEG, PNG, WEBP or HEIC/HEIF image."* 3. The upload action is aborted.
Priority / Frequency	High / High (executed multiple times daily per active user).
Special Requirements	1. Mobile-responsive camera capture support. 2. The backend shall normalise supported images to RGB JPEG, apply camera orientation and resize them to fit within 2048 × 2048 pixels while preserving aspect ratio before nutritional analysis.

2.2.1.9.2. Add Context Note

Use Case ID	MA - 2
Use Case Name	Add Context Note
Created By / Updated By	Matthew Wong
Date Created / Updated	23/09/2026
Actors	User
Description	The user inputs optional text details providing preparation context, hidden ingredients, or portion specifications.
Trigger	The user clicks or focuses on the context note text area.

Preconditions	1. The user is logged in. 2. The user is on the Meal Analysis page. 3. A meal image has been selected or staged.
Postconditions	The contextual string is captured and bound to the analysis request payload.
Normal Flow	1. The user taps or clicks the "Add notes/context (optional)" text field. 2. The user types specific preparation details (e.g., *"Cooked with 1 tbsp olive oil, dressing on the side"*). 3. The system stores the text string in a temporary client state. 4. The string is appended to the upcoming analysis request payload.
Alternative Flows	**2a. Field Left Empty:** 1. The user skips the field and submits the analysis without entering text. 2. The system sets the context note parameter to an empty string and continues.
Exceptions	**E1. Unsupported File Format:** 1. The user enters text exceeding the maximum limit (500 characters). 2. The system truncates input and displays a warning: *"Character limit exceeded (max 500 characters)."*
Priority / Frequency	Medium / Medium (used when meals have hidden oils, dressings, or sauces).
Special Requirements	Inline character counter showing remaining characters.

2.2.1.9.3. Analyze Calories & Macros

Use Case ID	MA - 3
Use Case Name	Analyze Calories & Macros
Created By / Updated By	Matthew Wong
Date Created / Updated	23/09/2026
Actors	User, AI Nutrition Analysis Service
Description	The system transmits the meal photo and context note to the AI service to extract estimated calories, macronutrients, and ingredient components.
Trigger	The user clicks the "Analyze Meal" button.
Preconditions	1. A valid image is staged (MA-1). 2. The user is authenticated.
Postconditions	A structured draft of total calories, macros (protein, carbs, fat), confidence score, and itemized ingredients is loaded into the UI.
Normal Flow	1. The user clicks "Analyze Meal". 2. The system sets the UI to a loading state with an animated spinner. 3. The frontend sends a multipart request with image bytes and context note to the backend (/api/nutrition/analyze). 4. The backend invokes the AI Nutrition Analysis Service using structured Pydantic schema enforcement. 5. The AI service processes visual features, estimates portion sizes, and calculates macros. 6. The AI service returns a structured JSON payload. 7. The backend passes the result to the frontend. 8. The frontend displays the editable meal preview card.
Alternative Flows	**6a. Non-Food Image Uploaded:** 1. The AI model detects no identifiable food items.

	2. The system displays an error banner: "No food detected. Please upload a clear photo of your meal." 3. The UI resets, prompting the user to try again.
Exceptions	**E1. AI Service Timeout or Network Error:** 1. The backend call to the AI service times out (> 10 seconds) or returns HTTP 5xx. 2. The system displays: *"Analysis failed due to a server error. Please try again or log manually."* 3. The user remains on the upload screen with the staged photo preserved.
Priority / Frequency	High / High (executed every time a meal photo is submitted).
Special Requirements	Strict server-side timeout of 10 seconds; structured JSON validation matching MealAnalysisResult schema.

2.2.1.9.4. Review Estimated Macros

Use Case ID	**MA - 4**
Use Case Name	Review Estimated Macros
Created By / Updated By	Matthew Wong
Date Created / Updated	13/09/2026

Actors	User
Description	The user reviews the AI-generated estimates of total calories, macro splits, and individual items to determine if corrections are required.
Trigger	Automatic display of the meal breakdown card upon completion of MA-3
Preconditions	MA-3 (Analyze Calories & Macros) completed successfully.
Postconditions	The user verifies the accuracy of the draft estimates.
Normal Flow	1. The system displays the dish title, total calories (kcal), and confidence indicator. 2. The system displays color-coded badges for total Protein (g), Carbohydrates (g), and Fat (g). 3. The system renders the itemized ingredient list with estimated gram weights and sub-macros. 4. The user inspects the values against their actual meal.
Alternative Flows	**Estimates Accepted As-Is:** 1. The user considers all values accurate without editing. 2. The user proceeds directly to MA-7 (Log Confirmed Meal).
Exceptions	None.
Priority / Frequency	High / High.
Special Requirements	Clear visual hierarchy; distinct color coding for macros (e.g., Protein: Blue, Carbs: Amber, Fat: Rose).

2.2.1.9.5. Edit Meal Breakdown

Use Case ID	**MA - 5**
Use Case Name	Edit Meal Breakdown
Created By / Updated By	Matthew Wong
Date Created / Updated	13/09/2026
Actors	User
Description	The user edits portion sizes, removes false positive ingredients, or manually adds missing items to adjust the meal breakdown.
Trigger	The user clicks on an ingredient weight field, delete icon, or "Add Item" button.
Preconditions	The meal breakdown card is displayed on screen (MA-4).
Postconditions	Updated ingredient data and proportionally recalculated meal totals are reflected in the UI.
Normal Flow	1. The user modifies the gram weight of an item (e.g., changing chicken breast from 150g to 200g). 2. The system recalculates that ingredient's macros in real time based on the new weight. 3. The system updates the total meal calories and macronutrient sums. 4. The updated totals are refreshed immediately in the UI.

Alternative Flows	**1a. Deleting an Incorrect Item:** 1. The user clicks the trash/delete icon next to an ingredient. 2. The system removes the item from the list. 3. The system subtracts its values from the meal totals. **1b. Adding a Missing Item:** 1. The user clicks "+ Add Item". 2. The user inputs ingredient name, estimated weight, and macros. 3. The system appends the row and updates overall totals.
Exceptions	**E1. Invalid Numeric Input:** 1. The user enters negative numbers or non-numeric characters in a weight/macro field. 2. The system rejects the keystroke and sets the field back to the previous valid integer.
Priority / Frequency	High / High (frequently used for portion tuning).
Special Requirements	Client-side portion scaling and draft calorie/macronutrient totals must update with < 50 ms latency without server roundtrips. These are review calculations only. On confirmation, FastAPI validates the submitted ingredients and ownership, and the server/database recomputes authoritative totals from the reviewed ingredients before returning the saved meal.

2.2.1.9.6. View Training Context Note

Use Case ID	**MA - 6**
Use Case Name	View Training Context Note

Created By / Updated By	Matthew Wong
Date Created / Updated	13/09/2026
Actors	User, AI Nutrition Analysis Service
Description	The user reads athletic guidance generated by the AI evaluating the meal's nutrient profile against their workout demands.
Trigger	Rendering of the analysis card following MA-3.
Preconditions	MA-3 (Analyze Calories & Macros) completed successfully.
Postconditions	The user is presented with contextual advice regarding recovery or fueling timing.
Normal Flow	1. The system extracts the training-context-note string from the analysis payload. 2. The system renders a highlighted callout box containing the advice (e.g., *"Contains 38g protein: optimal for muscle protein synthesis following your 140 TRIMP interval workout"*). 3. The user reviews the note to evaluate dietary recovery.
Alternative Flows	**No Context Generated:** 1. If the AI returns an empty context note, the system displays a default recovery reminder: *"Balanced meal for general endurance recovery."*
Exceptions	None.
Priority / Frequency	Medium / High.
Special Requirements	Callout container styled with an accent border (e.g., emerald/blue) and an athletic insight icon.

2.2.1.9.7. Log Confirmed Meal

Use Case ID	**MA - 7**
Use Case Name	Log Confirmed Meal
Created By / Updated By	Matthew Wong
Date Created / Updated	13/09/2026
Actors	User
Description	The user commits the finalized meal details to the database to update their daily nutritional log.
Trigger	The user clicks the "Confirm & Log to Profile" button.
Preconditions	1. The user has reviewed estimated macros (MA-4). 2. The user is logged into their account with a valid session.
Postconditions	A record is inserted into the public.meals table, daily totals update, and the form resets.

Normal Flow	1. The user verifies or selects the meal category (Breakfast, Lunch, Dinner, or Snack). 2. The user clicks "Confirm & Log to Profile". 3. The system sends a POST request with the finalized payload (name, category, total calories, macros, ingredient JSON) to the database. 4. The database executes the insert query under auth.uid(). 5. The system recalculates the user's daily consumed totals for the day. 6. The system displays a confirmation notification: *"Meal successfully logged!"*. 7. The UI clears the uploaded image and form, returning to the ready state.
Alternative Flows	**User Cancels Logging:** 1. The user clicks "Discard" or navigates away before confirming. 2. The system prompts: *"Are you sure you want to discard this meal log?"* 3. If confirmed, draft data is wiped without saving to the database.
Exceptions	**E1. Database Persistence Failure:** 1. The database connection fails or returns an authorization error. 2. The system displays: *"Unable to save meal log. Please try again."* 3. The draft data remains intact in the UI with the confirm button re-enabled for a retry.
Priority / Frequency	High / High (executed for every meal logged).

Special Requirements	Enforce Row-Level Security (RLS) so writes are strictly scoped to the authenticated user ID.

2.2.1.10. Gamification page

2.2.1.10.1.

Use Case ID	G - 1
Use Case Name	Perform Interactive Daily Check-in
Created By / Updated By	Yew Yi / Yew Yi
Date Created / Updated	13/09/2026
Actors	User
Description	The user submits the shared daily questionnaire to record muscle fatigue, mood and yesterday's diet adherence. A successful submission updates the check-in streak and awards check-in XP once per day.
Trigger	The user clicks the daily check-in button on the Homepage or Gamification page.
Preconditions	The user is securely authenticated and logged into their account.
Postconditions	The three ratings are saved for the user and current Singapore calendar date. The check-in streak and longest streak are updated, one check-in XP award is added, and both Home and Gamification reflect the saved check-in.
Normal Flow	1. The user navigates to the daily check-in interface. 2. The system displays the user's current consecutive check-in streak. 3. The system requests three required whole-number ratings from 1 to 5: muscle fatigue (1 = fresh, 5 = very sore), mood (1 = low, 5 = great), and yesterday's diet adherence (1 = off track, 5 = spot on). 4. The user completes the questionnaire and submits their responses. 5. After validating all three ratings, the system saves one check-in for the user and Singapore date. The streak starts at 1 for a first check-in, increases by 1 if the previous check-in was yesterday, and

	restarts at 1 after a missed day. An unbroken streak remains active when the latest check-in was today or yesterday; otherwise, the displayed current streak is 0. The longest streak is retained. 6. For the successful submission, the system awards 10 XP plus 2 XP for each streak day after the first, capped at 6 bonus days: day 1 earns 10 XP, day 2 earns 12 XP, and day 7 onward earns 22 XP. The interface displays the new total XP and completed check-in state.
Alternative Flows	If the user has already checked in on the current Singapore date, the system displays “Checked in for today” and the current streak. Home disables its check-in button. Repeated or concurrent submissions return an already-checked-in response and add no check-in, streak increment or XP award.
Exceptions	If the database is unreachable, the system displays an error message alerting the user that their check-in could not be saved and prompts them to try again later.
Priority / Frequency	High / Daily
Special Requirements	All three ratings must be supplied before submission; invalid or incomplete input is rejected without a check-in or XP award. The backend determines the day using Singapore time (UTC+8) and enforces one check-in per user per date, with one XP award per check-in. Sleep hours are entered separately on the Training page and do not complete the daily questionnaire or earn check-in XP; stress is not a questionnaire input.

2.2.1.10.2.

Use Case ID	G - 2
Use Case Name	Earn and View Experience Points (XP)
Created By / Updated By	Yew Yi / Yew Yi
Date Created / Updated	13/09/2026
Actors	User

Description	The system awards users Experience Points (XP) upon successful completion of training routines, daily check-ins, and consistent meal logging, and users can view their total accumulated XP.
Trigger	The user successfully completes a training routine, a daily check-in, or logs a meal.
Preconditions	The user is authenticated and logged into the application.
Postconditions	The user's XP total is incremented in the database and updated on the interface.
Normal Flow	1. The user performs a qualifying action (e.g., completes a training routine, check-in, or meal log). 2. The system verifies the successful completion of the action. 3. The system calculates the appropriate XP reward based on the action. 4. The system updates the user's total accumulated XP in the database. 5. The Gamification interface displays the newly updated XP total to the user.
Alternative Flows	If the user navigates directly to the Gamification page without triggering a new XP event, the system simply retrieves and displays the current accumulated XP without performing new calculations.
Exceptions	If a tracked activity fails to pass validation, the system holds the XP in a pending state and alerts the user to re-sync their data.
Priority / Frequency	High / Multiple times per day.
Special Requirements	The application shall provide fast response times by optimizing code so that XP updates feel instantaneous to the user

2.2.1.10.3.

Use Case ID	G - 3
Use Case Name	View and Compare Dynamic Leaderboards

Created By / Updated By	Yew Yi / Yew Yi
Date Created / Updated	13/09/2026
Actors	User
Description	The user views an interactive leaderboard to compare their performance against friends based on XP, running mileage, and streak consistency.
Trigger	The user navigates to the leaderboard section of the Gamification page.
Preconditions	The user is logged in and has added at least one friend via the Friend page.
Postconditions	The user is presented with accurate, up-to-date rankings of themselves and their friends.
Normal Flow	1. The user requests to view the leaderboard. 2. The system fetches the user's data and their friends' data (XP, running mileage, streak consistency). 3. The system sorts the data dynamically to determine rankings. 4. The system displays the interactive leaderboard, allowing the user to compare their ranking directly with friends.
Alternative Flows	The user can toggle between different filtering metrics, changing the leaderboard view from accumulated XP to running mileage or streak consistency.
Exceptions	If the user has zero friends, the system displays an empty state illustration and provides a direct link to the "Add Friend" interface.
Priority / Frequency	Medium / As desired by the user
Special Requirements	The database operations for fetching the leaderboard must be organized into separate layers from the UI to enhance testability

2.2.1.10.4.

Use Case ID	G - 4
Use Case Name	Manage and Participate in Shared Challenges

Created By / Updated By	Yew Yi / Yew Yi
Date Created / Updated	13/09/2026
Actors	User
Description	The user interacts with the system to create new shared fitness challenges, invite friends to them, or participate in existing ones.
Trigger	The user selects the option to create a challenge or accepts an invitation to join one.
Preconditions	The user is securely authenticated.
Postconditions	A new challenge is instantiated in the system, or the user is successfully enrolled in an active challenge.
Normal Flow	1. The user initiates the creation of a shared challenge. 2. The system prompts the user to define the challenge parameters. 3. The user selects friends to invite to the newly created challenge. 4. The system sends invitations to the selected friends. 5. Invited friends accept the invitation to participate in the challenge. 6. The system tracks all participants' progress dynamically against the challenge goals
Alternative Flows	A user receives an invitation to a shared challenge, reviews the goal metric, and clicks "Decline" instead of participating.
Exceptions	If a user attempts to invite another user who is not on their verified friends list, the system blocks the action and displays a validation error.
Priority / Frequency	Medium / Weekly or Monthly
Special Requirements	The challenge tracking logic must be designed to handle an increasing number of concurrent users and requests without performance degradation.

3. Constraints

3.1. Scalability

The system must handle an increasing volume of users, requests and data without experiencing performance degradation

3.2. Data and Function Mapping

Integrating external data sources like strava and samsung health requires secure token management and proper API mapping to ensure seamless synchronization across user profiles and training logs

3.3. Proprietary hardware and software

The application requires a device which can connect to the internet, such as a Desktop, laptop or mobile device.

Additional hardware that is not compulsory is smartwatches with integrated heart rate sensors, which supply additional heart rate health data to the Strava application.

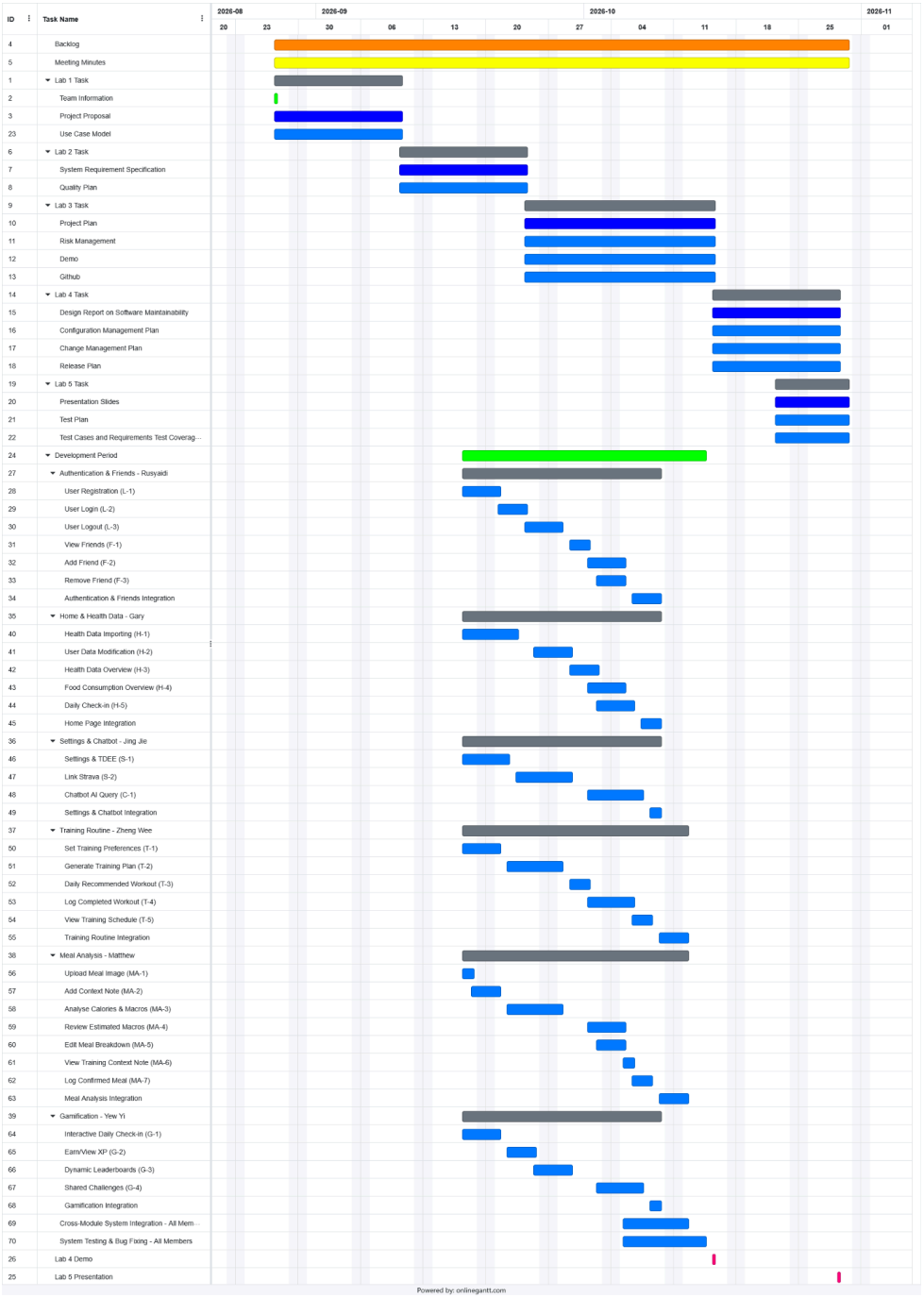
3.4. Batch updates vs. (close) Real-time updates

There are no batch updates for our application. All updates happen and are calculated in real time.

Health data	User health data is updated immediately after importing from health services or updating manually
Training data	Training data is updated immediately after importing from Strava.
Health statistical data	User health data are forwarded to an Artificial Intelligence Agent for analysis and return statistical health data immediately after the agent is done processing.
Training statistical data	User training data are forwarded to an Artificial Intelligence Agent for analysis and return curated training plans for the user.

3.5. Project Schedule

There is a nine-week timeframe to implement a prototype system of a fitness tracker from project commencement on Wednesday of week 3 (27 August 2026) to the end of week 11 (Sunday, 1st November 2026)



Online Gantt 20261104.pdf

Task	Start	End	Duration
Backlog	27 August 2026	30 October 2026	65 Days
Meeting Minutes	27 August 2026	30 October 2026	65 Days
Lab 1 Tasks			
Team Information	27 August 2026	27 August 2026	1 Day
Project Proposal	27 August 2026	10 September 2026	15 Days
Use Case Model	27 August 2026	10 September 2026	15 Days
Lab 2 Tasks			
System Requirement Specification (SRS)	10 September 2026	24 September 2026	15 Days
Quality Plan	10 September 2026	24 September 2026	15 Days
Lab 3 Tasks			
Project Plan	24 September 2026	15 October 2026	22 Days
Risk Management	24 September 2026	15 October 2026	22 Days
Demo (Preparation)	24 September 2026	15 October 2026	22 Days
Github	24 September 2026	15 October 2026	22 Days
Lab 4 Tasks			
Design Report on Software Maintainability	15 October 2026	29 October 2026	15 Days
Configuration Management Plan	15 October 2026	29 October 2026	15 Days
Change Management Plan	15 October 2026	29 October 2026	15 Days

Release Plan	15 October 2026	29 October 2026	15 Days
Lab 5 Task			
Presentation Slides	22 October 2026	29 October 2026	8 Days
Test Plan	29 October 2026	07 November 2026	10 Days
Test Cases and Requirements Test Coverage Report	29 October 2026	04 November 2026	7 Days
Development Period	17 September 2026	15 October 2026	29 Days
Lab 4 Demo	15 October 2026	15 October 2026	1 Day
Lab 5 Presentation	29 October 2026	29 October 2026	1 Day

4. Operational Requirements

4.1. Help Desk Support

Support for the application will be provided by the project team. Users can report issues regarding data import failures from health data, Linking errors from Strava, account access issues or incorrect meal estimates. These will be reported through an in application feedback form or email to the project team. Reported issues will be logged as tickets in the team JIRA project.

4.2. Application Services and Technical support

The project team's developers will have access to the source code on the Github Repository to address bugs and implement enhancements. Through the Supabase dashboard, it will provide database monitoring, authentication, management and access to system logs which will be settled by the release engineer if there are occurrences of failed builds, deployment issues and service outages.

4.3. Administration Features

The Application will provide varying system access levels. For a normal user, they can typically only view and modify their own profile, training/nutrition and personal data in general. Going a level higher to admins, they will be able to view system logs, monitor API usage against quotas and disable accounts in misuse. However, no admin can view another user's daily check in responses or readiness score.

4.4. System Interface independent of VRU

The web application will remain operational when an external service is unavailable. If certain APIs cannot be reached i.e. Strava API, data which is previously imported will remain viewable and the user can continue to log their sessions manually. If the Gemini API is unavailable, the meal logging portion will fallback to manual entry, with the chatbot displaying an error without blocking the full application. Core features must not depend on any external service being online.

4.5. System routine back up

Our frontend is hosted on Vercel, while our backend is hosted using Render and our database using Supabase.

Frontend: Vercel offers 19 autonomous cloud regions, with each region maintaining full metadata copies.

Backend: Render offers recovery and backups through point-in-time recovery, allowing us to restore our database to a previous state in the past, preventing and recovering any accidental data loss.

Database: Supabase offers support for users for Automated point-in-time recovery (PTR) and daily snapshots in the event of database failure.

4.6. Audit Trail

The system records an audit trail for all actions that the user has done or any action that affects the user's data. Each record will capture what action has been taken and the time at which the action occurred. Types of audited actions include profile and fitness goal changes, training plan generation and recalculation, meal logging and session logging. Audit records will be retained for the duration of the project and are used to trace defects reported during testing.

5. Functional Requirements

5.1. Login/Registration page

5.1.1. **User Registration:** The system shall allow users to create an account.

5.1.2. **User Login:** The system shall authenticate users using their credentials.

5.1.3. **User Logout:** The system shall allow authenticated users to log out of their account

5.2. Friend page

5.2.1. **View Friends:** The system shall allow users to view their current friends.

5.2.2. **Add Friend:** The system shall allow users to search for and add other users as friends.

5.2.3. **Remove Friend:** The system shall allow users to remove existing friends.

5.3. Home page

5.3.1. **Health data importing:** The system shall allow users to import their personal health and fitness data, such as height and weight, from external services such as Samsung Health or Apple Health.

5.3.2. **Training overview:** The dashboard on the homepage shall display a visual summary of the user's training activity, such as total mileage covered during runs and average calories consumed over a period of time.

5.3.3. **Daily check-in: The Home dashboard shall display saved check-in dates and the current check-in streak. Its button shall open the shared G-1 questionnaire on the leaderboard page and shall display "Checked in" and be disabled after today's submission. Opening the questionnaire alone shall not change the streak or award XP.**

5.4. Settings page

Profile & TDEE Engine

5.4.1. **Biological metrics:** The application must allow users to input their biological metrics: age (int, years), weight (float, kg), height (float, cm), biological sex (string, male/female)

5.4.2. **Activity levels:** The application must allow users to select their current activity level (Sedentary, Lightly Active, Moderately Active, Very active, Extra Active) and their fitness goal (Deficit, Maintenance, Surplus).

5.4.3. **Health statistics:**

5.4.3.1. Upon submission of user's profile metrics, the application must calculate their Basal Metabolic Rate (BMR) and Total Daily Energy Expenditure (TDEE) using Mifflin-St Jeor equation

5.4.3.2. The application must calculate the daily macronutrient (grams of protein, Carbs, Fats) based on computed TDEE and their activity level

5.4.4. **Strava OAuth Integration:**

5.4.4.1. The application must provide a "Link Strava" UI that redirects unauthenticated clients to the official Strava OAuth 2.0 authorization portal.

5.4.4.2. The backend shall receive a temporary access_token and persistent refresh_token via HTTP POST request to Strava API

5.5. Meal analysis page

5.5.1. Media Capture & Ingestion:

5.5.1.1. The system shall allow users to upload meal images via local file selection.

5.5.1.2. The system shall accept JPEG, PNG, WEBP and HEIC/HEIF images up to 10 MB. The backend shall validate the image content and convert supported images to JPEG before nutritional analysis; HEIC/HEIF conversion shall occur on the server.

5.5.1.3. The user shall have the option to attach a short supplementary text description or voice note transcript (e.g., "cooked with 1 tbsp butter", "whole milk used") before initiating the analysis.

5.5.2. AI Vision & Multimodal Extraction:

5.5.2.1. The backend shall extract an itemized breakdown of food components from the uploaded image.

5.5.2.2. For each identified item, the system shall estimate:

5.5.2.2.1. Component name (e.g., Grilled Chicken Breast)

5.5.2.2.2. Estimated portion weight in grams (g)
5.5.2.2.3. Total calories (kcal)
5.5.2.2.4. Macronutrient profile: Protein (g), Carbohydrates (g), and Fat (g)

5.5.2.3. The system shall aggregate itemized values into total meal calories and macronutrient totals.

5.5.2.4. The system shall provide an inference confidence score (0.0 to 1.0) representing visual certainty.

5.5.2.5. The model shall generate contextual athletic feedback evaluating the meal against endurance energy demands (e.g., glycogen replenishment, protein synthesis timing relative to workout load).

5.5.3. Human-in-the-Loop Review & Modifications

5.5.3.1. The user interface shall display the estimated macronutrients and itemized list in an editable preview state prior to database persistence.

5.5.3.2. The user shall be able to:

5.5.3.2.1. Adjust serving sizes/weights (which automatically recalculates the item's and total meal's macros proportionally).
5.5.3.2.2. Add missed food items manually.
5.5.3.2.3. Delete incorrectly identified ingredients.
5.5.3.2.4. Reclassify the meal category (Breakfast, Lunch, Dinner, Snack).

5.5.4. Daily Food Logging & Aggregation

5.5.4.1. Upon user confirmation, the system shall write the finalized meal log to the database (meals table) linked to the authenticated user's ID.

5.5.4.2. The system shall update the athlete’s rolling daily total for calories, protein, carbohydrates, and fats.

5.5.4.3. The system shall compute the net energy balance against daily energy expenditure: Net Energy = Calories In - (BMR + Strava Active Calorie Burn)

5.6. Chatbot page

5.6.1. Conversational Interface

5.6.1.1. The application must provide a text-based input and submit button for users to query the chatbot using natural language regarding their fitness journey

5.6.1.2. The chatbot interface must display a scrollable, chronological history of active chats.

5.6.1.3. The chatbot interface must visually differentiate between user prompts and generated responses

5.6.2. Context

5.6.2.1. The backend service must append the authenticated user's data, such as TDEE, fitness goals, logged meals, and exercise history, as hidden context to the LLM prompt

5.6.2.2. The chatbot must analyse user feedback and propose actionable insights into their training routine

5.7. Training routine page

5.7.1. Training plan generation

5.7.1.1. **Training preferences:** The application must allow users to specify their training days per week (int, 1 to 7), preferred session duration (int, 15 to 180 minutes), preferred activity types (run, gym, cycling, bodyweight), rest days, and up to 10 exam dates falling within the next 730 days. The number of training days must not exceed the number of non-rest days in the week.

5.7.1.2. **Plan generation**: The system shall generate a personalised training plan of 2 to 16 whole weeks from the user's profile metrics, fitness goal, training preferences and training history. For this requirement, training history means sessions the user logged in Fitra and sessions imported from a health export file. Activities retrieved from Strava are excluded, as required by 5.7.4. Every fourth week of the plan shall be a deload week.

5.7.1.3. **Weekly progression rate:** Where the user has at least six complete weeks of their own logged training, the default weekly increase shall be derived from that history and capped at 10%. Otherwise the default shall be 5%. The Training page shall display the rate in use, how it was derived, and shall state that neither Strava activities nor readiness data were used to derive it.

5.7.1.4. **Progressive overload constraint**: Weekly training volume is measured in load points, derived from the duration and intensity of each planned session and summed over the week. The generated plan shall increase weekly training volume by no more than 10% between consecutive weeks. The user may configure the weekly increase anywhere from 0% to 10%. 10% is a hard safety cap and the user cannot set a value above it; a request above 10% shall be refused with the reason. Deload weeks are exempt, as they reduce volume rather than increase it.

5.7.2. Readiness scoring

5.7.2.1. Intensity classification: The system shall compute a daily readiness score from 0 to 100 and classify the day as deload below 45, light from 45 to 69 inclusive, and push at 70 and above.

5.7.2.2. **Readiness inputs:** The readiness score shall be computed from the user's daily check-in ratings for muscle fatigue, mood and diet adherence, the sleep hours entered on the Training page (0 to 16 hours), the user's own logged training history, and any exam date falling within the next 7 days. Activities retrieved from Strava shall not be used, as required by 5.7.4.

5.7.2.3. **One-directional adjustment:** Readiness may lower the intensity of a planned session but shall never raise it, so that the periodisation of the plan is preserved. A planned deload session shall never be raised.

5.7.3. Daily recommended workout

5.7.3.1. **Workout display**: The system shall display the recommended workout for the current day, including exercises or target distance, target intensity, and estimated session duration.

5.7.3.2. **User response**: The system shall allow the user to accept, swap, or skip the recommended workout for that day.

5.7.3.3. **Session logging**: The system shall record completed workouts back into the user's training history for use in subsequent readiness calculations and plan adaptation. A session logged at less than 80% of its planned load shall be recorded as Partial, and at 80% or above as Completed.

5.8. Gamification page

5.8.1. **Experience Point (XP) System:** The system shall award users Experience Points (XP) upon successful completion of training routines, daily check-ins, and consistent meal logging.

5.8.2. **Dynamic Leaderboards:** The system shall display an interactive leaderboard ranking the user against their friends based on accumulated XP, running mileage, and streak consistency.

5.8.3. **Shared User Challenges:** The system shall allow users to create, invite friends to, and participate in shared fitness challenges.

5.8.4. **Interactive Daily Check-in: Home and Gamification shall use one shared questionnaire with required 1–5 ratings for muscle fatigue, mood and yesterday's diet adherence. A successful submission shall count once per user per Singapore calendar date (UTC+8), update the check-in streak and award 10 XP plus a streak bonus of 2 XP per day after day 1, capped at 22 XP from day 7. Consecutive-day submissions extend the streak; a missed day breaks it and the next submission starts at 1. Same-day repeats shall not change the streak or award XP. Sleep hours shall be recorded separately on the Training page.**

6. Non-functional requirements

6.1. Performance

The application shall provide fast response times by optimising code where possible.

6.2. Scalability

The system shall be designed to handle an increasing number of users, requests and data without performance degradation.

6.3. Maintainability

The application shall separate UI components and draft-state calculation helpers in the Next.js frontend from authenticated feature rules and persistence in FastAPI and Supabase. MA-5 portion scaling and draft calorie/macronutrient totals shall run client-side for immediate feedback. FastAPI shall validate confirmed ingredient data and ownership, and the server/database shall derive persisted totals from those ingredients. Application, readiness and integration modules are logical layers within one FastAPI service.

6.4. User experience

All actions performed by the user shall have a form of feedback to let the user be aware that processes are happening in the background.

6.5. Security

The system shall securely authenticate users and protect user account information. It shall also restrict access to authenticated features to logged-in users.

7. Input Requirements

7.1. User identifier key and user access

Each user is assigned a unique identifier upon registration , which maps to all their personal profiles , health statistics and training records within the system. Users authenticate their credentials to access the online platform and accounts can be managed or deactivated when necessary.

7.2. Health and training data

Biological metrics , fitness goals and training preferences such as age , weight , height , activity level and weekly training volume are inputted by the user or imported from external services such as strava.

7.3. Action Codes

All interactive action inputs , such as capturing meal images , writing short context notes , adjusting portion weights , logging daily check ins , or accepting recommended workouts are available online for reference and system processing.

8. Process Requirements

The following process requirements define the procedures and technical considerations that the health tracker system shall follow to ensure reliable, secure and maintainable software operations.

8.1. User Authentication

The system shall implement user authentication through Supabase Authentication to ensure that only registered users can access protected features.

8.2. Data integrity

The system shall maintain the accuracy, consistency and reliability of user and health -related data stored in the database

8.3. Data Validation and Error Handling

Data error from the user's end and from the back-end database-processing end must be gracefully handled. There will be data validation and error-handling routines as part of the online registration system.

8.4. API and Database Communication

The system shall maintain reliable communication between the frontend and backend cases and database.

8.5. Performance and Availability

The system shall provide reasonable response times and remain functional during normal usage.

8.6. Data Security

The system shall protect user information from unauthorised access and disclosure.

9. Output Requirements

9.1. Transaction summary and confirmation

User view summaries of their completed actions for a specific session, such as saved meal logs, recorded workouts or submitted daily check-ins, while the database records all successfully processed entries.

9.2. Exception reports

The system highlights special conditions or anomalies such as low AI vision confidence during image analysis or extreme deviations in a user’s daily energy expenditure.

9.3. Registration Reports and summaries

Athletes view consolidated visual summaries of their rolling calorie intake , macronutrient totals and weekly training workloads through interactive charts.

10. Hardware Requirements

10.1. Network

A stable wired or wireless network infrastructure connects the platform to extern cloud services and real time APIs

10.2. Client Computers

Users access the platform on desktop computers, tablets, and mobile devices equipped with modern web browsers

10.3. Server Infrastructure

Cloud-hosted servers run the backend services, handling API requests and database management

10.4. Production support Systems

The production environment utilizes reliable cloud hosting, automated data backups and secure hosting infrastructure to maintain application uptime.

11. Software Requirements

The following software requirements specify the software platforms, development tools and services required for the development and operation of the Health Tracker System.

11.1. Operating System

The development environment shall support Windows, macOS, or Linux operating systems.

11.2. Client Application

Any modern web browser that supports JavaScript, such as Google Chrome.

11.3. Frontend

The system frontend shall be developed using Next.js, Rechart and Lucide React, TypeScript and Tailwind CSS.

11.4. Backend

The backend shall use Python FastAPI with Uvicorn. Node.js shall host the Next.js application; it is not a separate application backend or inference service.

11.5. Database and Authentication Services

The system shall use Supabase for database management and user authentication.

12. Deployment Requirements

12.1. System architecture overview

The repository deployment has two application services, frontend and backend, plus managed Supabase, Google Gemini and Strava services. Docker Compose also provisions a Redis container, but the active API does not enqueue jobs or start a queue consumer. The application does not deploy a separate inference server or background-worker service.

- Frontend (frontend, port 3000): Next.js 15 on Node.js 20 serves the web interface, manages Supabase sessions and OAuth callback navigation, and performs draft nutrition calculations. Browser and Next.js server requests call the configured FastAPI URL; browser Supabase access uses the user's session and database RLS.
- Backend (backend, port 8000): Python 3.12 FastAPI/Uvicorn exposes profile, nutrition, check-in, gamification, training, chat and Strava APIs. It verifies authenticated requests, applies feature rules, accesses Supabase, normalises meal images, runs readiness scoring in-process and calls Gemini or Strava when requested.
- Redis (redis, port 6379): Docker Compose provisions Redis and waits for its health check before starting the backend. Redis is not used by the currently mounted API routes. The legacy queue/worker code is not started by FastAPI or Compose and is not an active Strava ingestion or calculation service.
- External Managed Services:
    - Database & Auth: Supabase Managed PostgreSQL instance with Row-Level Security (RLS) and Supabase Auth
    - External telemetry: Strava OAuth and v3 REST API calls for account linking, on-demand activity retrieval and saving selected activities to the training schedule. The current FastAPI application exposes no Strava webhook receiver.

- External AI: FastAPI calls the Google Gemini API through the Google GenAI SDK for meal analysis and AI coaching. The nutrition model is selected by backend configuration. The readiness model runs locally within FastAPI with a rule-based fallback.

12.2. Hardware requirements

Tier	Component / Container	Runtime / Technology	Role & Services
Client Tier	Athlete Workstations / Mobile	Modern Web Browser (Chrome, Safari, Edge)	Progressive web interface, camera/file capture, real-time dashboard rendering.
Frontend Tier	nextjs-frontend	Node.js 20+ (Alpine Docker image), Next.js 15	Next.js rendering, session handling, OAuth callbacks and UI logic; no separate Node.js backend service.
Backend Tier	Fastapi-backend	Python 3.12-slim, Uvicorn ASGI server	Authenticated REST APIs, feature rules, image processing, in-process readiness scoring, Supabase access and Gemini/Strava integration.
Provisioned Redis	redis	Redis 7-Alpine	Included by Docker Compose; no

			active API producer or worker consumer. It performs no current ingestion or calculation work.
Database Tier	Cloud-Hosted Supabase	PostgreSQL 15+	Managed PostgreSQL persistence for user profiles, training records, check-ins, gamification, meal ingredients and chat history.
AI Services	Google Gemini API	External API via Google GenAI SDK	Multimodal meal vision analysis and contextual sports physiology advice generation.

12.3. Deployment prerequisites & configuration

- Containerisation & Orchestration:
    - Docker Engine v24+ & Docker Compose v2+ installed on target host
    - Multi-stage Docker builds configured for minimal container image sizes (Alpine/ Slim base images)
- Environment Secret Management (.env)
    - Supabase: SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY

    - Strava OAuth: STRAVA_CLIENT_ID and STRAVA_CLIENT_SECRET. STRAVA_VERIFY_TOKEN remains a legacy configuration field; no webhook receiver is mounted.
    - AI provider: GOOGLE_GENERATIVE_AI_API_KEY; GEMINI_NUTRITION_MODEL selects the meal-analysis model.
    - Service URLs: NEXT_PUBLIC_BACKEND_URL points to the browser-reachable FastAPI address; FRONTEND_URL configures the allowed frontend origin. Compose sets REDIS_URL=redis://redis:6379/0 for the provisioned Redis container.
- Database Setup (Supabase PostgreSQL):
    - Apply the release's approved Supabase migrations for the existing team database, including the nutrition, training and gamification functions and tables. Do not initialise the shared database from the legacy supabase/schema.sql scaffold.
    - Verify Row-Level Security (RLS) policies are active so athletes can only read and write their own records.
    - Configure the Strava developer application's authorised callback to the Next.js /api/auth/callback route; the callback exchanges the code through FastAPI /strava/exchange using the signed-in session.

12.4. Deployment artifacts & procedure

- Deployment Artifacts:
    - frontend/Dockerfile: Multi-stage Node.js container build.
    - backend/Dockerfile: Lightweight Python ASGI container image.
    - docker-compose.yml: Local/VPS container orchestration definition.
- Step-by-Step Deployment Procedure:
    1. Clone the repository to the deployment server.
    2. Populate production environment variables in .env.
    3. Execute database migrations against the Supabase database instance.

4. Launch container suite using Docker Compose:
    - docker compose up --build -d
5. Configure reverse proxy (Nginx, Caddy, or Cloudflare Tunnel) with SSL certificates directing:
    - Frontend traffic to port 3000.
    - Backend API traffic to port 8000.
6. Connect Strava through Settings or Training, then retrieve activities on demand and save selected activities through the Training page. The current deployment does not require a Strava webhook subscription.
    - No worker process is launched by the current backend entry point or Docker Compose definition. Redis queue processing and webhook ingestion must not be treated as active deployment components.
7. Run a /health connectivity check, then exercise authenticated training/readiness, meal analysis and chat through the UI or their current API routes. Verify Supabase access and Gemini configuration; no separate inference or queue worker needs to be started.

.